

Prac 1:

```
#include <iostream>

using namespace std;

// Class definition
class ATM {
    private:
        double balance; // Encapsulated data (private)
    public:
        // Constructor
        ATM(double initialBalance) {
            if (initialBalance >= 0)
                balance = initialBalance;
            else {
                cout << "Invalid initial balance. Setting to 0.\n";
                balance = 0;
            }
        }

        // Balance Inquiry
        void getBalance() const {
            cout << "\nCurrent Balance: " << balance << endl;
        }

        // Deposit function
        void deposit(double amount) {
            if (amount > 0) {
                balance += amount;
                cout << "Deposited " << amount << " successfully.\n";
            } else {
                cout << "Invalid deposit amount!\n";
            }
        }

        // Withdrawal function
        void withdraw(double amount) {
```

```

        if (amount <= 0) {
            cout << "Invalid withdrawal amount!\n";
        } else if (amount > balance) {
            cout << "Insufficient balance!\n";
        } else {
            balance -= amount;
            cout << "Withdrawn " << amount << " successfully.\n";
        }
    }
};

// Main function
int main() {
    double initialBalance;
    cout << "Enter initial balance:";
    cin >> initialBalance;
    ATM user(initialBalance); // Constructor called
    int choice;
    double amount;

    do {
        cout << "\n===== MINI ATM MENU =====";
        cout << "\n1. Balance Inquiry";
        cout << "\n2. Deposit";
        cout << "\n3. Withdraw";
        cout << "\n4. Exit";
        cout << "\nEnter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: user.getBalance();
                break;
            case 2: cout << "Enter deposit amount: ";
                cin >> amount;

```

```
        user.deposit(amount);  
        break;  
    case 3: cout << "Enter withdrawal amount: ";  
            cin >> amount;  
            user.withdraw(amount);  
            break;  
    case 4: cout << "Thank you for using the ATM!\n";  
            break;  
    default:cout << "Invalid choice! Please try again.\n";  
            }  
} while (choice != 4);  
return 0;  
}
```

Parc 2:

```
#include <iostream>

#include <sstream>

#include <string>

#include <stdexcept>

using namespace std;


// -----
// Inline arithmetic functions
// -----

inline double add(double a, double b) { return a + b; }

inline double subtract(double a, double b) { return a - b; }

inline double multiply(double a, double b) { return a * b; }

inline double divide(double a, double b) { return b != 0 ? a / b : throw runtime_error("Division by
zero!"); }


// -----
// Function overloading examples
// -----

double calculate(double a, char op, double b) {

    switch (op) {

        case '+': return add(a, b);

        case '-': return subtract(a, b);

        case '*': return multiply(a, b);

        case '/': return divide(a, b);

        default: throw invalid_argument("Invalid operator!");

    }

}


// Overloaded function for integer inputs

int calculate(int a, char op, int b) {

    switch (op) {

        case '+': return a + b;
```

```

        case '-': return a - b;
        case '*': return a * b;
        case '/': return b != 0 ? a / b : throw runtime_error("Division by zero!");
        default: throw invalid_argument("Invalid operator!");
    }
}

// -----
// Parse and evaluate expression
// -----
double parseExpression(const string& expr) {
    stringstream ss(expr);
    double num1, num2;
    char op;

    ss >> num1 >> op >> num2;

    if (ss.fail()) throw invalid_argument("Invalid expression format!");

    // Use overloaded function
    return calculate(num1, op, num2);
}

// -----
// Main Function
// -----
int main() {
    cout << "=== Basic Calculator ===" << endl;
    cout << "Supports +, -, *, / operations between two numbers." << endl;
    cout << "Example: 12.5 * 3" << endl;

    string expression;

```

```
while (true) {  
    cout << "\nEnter expression (or type 'exit' to quit): ";  
    getline(cin, expression);  
  
    if (expression == "exit") break;  
  
    try {  
        double result = parseExpression(expression);  
        cout << "Result: " << result << endl;  
    }  
    catch (const exception& e) {  
        cerr << "Error: " << e.what() << endl;  
    }  
}  
  
cout << "Calculator closed." << endl;  
return 0;  
}
```

Prac 3:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Function to calculate grade using conditionals
```

```
char calculateGrade(float marks) {
```

```
    if (marks >= 90) return 'A';
```

```
    else if (marks >= 80) return 'B';
```

```
    else if (marks >= 70) return 'C';
```

```
    else if (marks >= 60) return 'D';
```

```
    else return 'F';
```

```
}
```

```
// Function to display menu
```

```
void displayMenu() {
```

```
    cout << "\n===== STUDENT GRADING SYSTEM =====" << endl;
```

```
    cout << "1. Enter Student Record" << endl;
```

```
    cout << "2. Display All Records" << endl;
```

```
    cout << "3. Exit" << endl;
```

```
    cout << "Enter your choice: ";
```

```
}
```

```
int main() {
```

```
    const int MAX = 100;
```

```
    string names[MAX];
```

```
    float marks[MAX];
```

```
    char grades[MAX];
```

```
    int count = 0;
```

```
    int choice;
```

```

do {
    displayMenu();
    cin >> choice;

    switch (choice) {
        case 1: {
            if (count >= MAX) {
                cout << "Record limit reached!" << endl;
                break;
            }

            cout << "Enter student name: ";
            cin.ignore(); // Clear input buffer
            getline(cin, names[count]);

            cout << "Enter marks (0 - 100): ";
            cin >> marks[count];

            // Conditional structure to validate marks
            if (marks[count] < 0 || marks[count] > 100) {
                cout << "Invalid marks! Please enter between 0 and 100." << endl;
            } else {
                grades[count] = calculateGrade(marks[count]);
                cout << "Grade assigned: " << grades[count] << endl;
                count++;
            }
            break;
        }

        case 2: {
            if (count == 0) {
                cout << "No records to display!" << endl;
            }
        }
    }
}

```



```

    } else {
        cout << "\n===== STUDENT RECORDS =====" << endl;
        for (int i = 0; i < count; i++) {
            cout << i + 1 << ". " << names[i]
                << " - Marks: " << marks[i]
                << " - Grade: " << grades[i] << endl;
        }
    }
    break;
}

case 3:
    cout << "Exiting program..." << endl;
    break;

default:
    cout << "Invalid choice! Please try again." << endl;
}
} while (choice != 3); // Looping construct

cout << "Program terminated successfully." << endl;
return 0;
}

```

Prac 4:

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex {
```

```
private:
```

```
    float* real;
```

```
    float* imag;
```

```
public:
```

```
    // Constructor
```

```
    Complex(float r = 0, float i = 0) {
```

```
        real = new float;
```

```
        imag = new float;
```

```
        *real = r;
```

```
        *imag = i;
```

```
    }
```

```
    // Copy Constructor (important for dynamic memory)
```

```
    Complex(const Complex& c) {
```

```
        real = new float;
```

```
        imag = new float;
```

```
        *real = *c.real;
```

```
        *imag = *c.imag;
```

```
    }
```

```
    // Operator Overloading for Addition
```

```
    Complex operator + (const Complex& c) {
```

```
        Complex temp;
```

```
        *temp.real = *real + *c.real;
```

```

        *temp.imag = *imag + *c.imag;

    return temp;
}

// Operator Overloading for Subtraction
Complex operator - (const Complex& c) {
    Complex temp;
    *temp.real = *real - *c.real;
    *temp.imag = *imag - *c.imag;
    return temp;
}

// Operator Overloading for Multiplication
Complex operator * (const Complex& c) {
    Complex temp;
    *temp.real = (*real * *c.real) - (*imag * *c.imag);
    *temp.imag = (*real * *c.imag) + (*imag * *c.real);
    return temp;
}

// Display Function
void display() {
    cout << *real << " + " << *imag << "i" << endl;
}

// Destructor
~Complex() {
    delete real;
    delete imag;
}
};

```

```
int main() {  
    Complex *c1 = new Complex(4, 5);  
    Complex *c2 = new Complex(2, 3);  
  
    Complex sum = *c1 + *c2;  
    Complex diff = *c1 - *c2;  
    Complex prod = *c1 * *c2;  
  
    cout << "First Complex Number: ";  
    c1->display();  
  
    cout << "Second Complex Number: ";  
    c2->display();  
  
    cout << "Addition: ";  
    sum.display();  
  
    cout << "Subtraction: ";  
    diff.display();  
  
    cout << "Multiplication: ";  
    prod.display();  
  
    delete c1;  
    delete c2;  
  
    return 0;  
}
```

Prac 5:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Student {
```

```
private:
```

```
    int roll;
```

```
    string name;
```

```
    float marks;
```

```
    static int count;
```

```
public:
```

```
    void input(int r, string n, float m) {
```

```
        roll = r;
```

```
        name = n;
```

```
        marks = m;
```

```
    }
```

```
    void display() {
```

```
        cout << "Roll No: " << roll
```

```
        << ", Name: " << name
```

```
        << ", Marks: " << marks << endl;
```

```
    }
```

```
    int getRoll() { return roll; }
```

```
    friend void addStudent(Student s[], int &n);
```

```
    friend void deleteStudent(Student s[], int &n, int roll);
```

```
    friend void searchStudent(Student s[], int n, int roll);
```

```
};
```

```
int Student::count = 0;
```

```
void addStudent(Student s[], int &n) {
```

```
    int r;
```

```
    string nm;
```

```
    float m;
```

```
    cout << "\nEnter Roll No: ";
```

```
    cin >> r;
```

```
    cout << "Enter Name: ";
```

```
    cin >> nm;
```

```
    cout << "Enter Marks: ";
```

```
    cin >> m;
```

```
    s[n].input(r, nm, m);
```

```
    n++;
```

```
    Student::count++;
```

```
    cout << "Student Added Successfully!\n";
```

```
}
```

```
void deleteStudent(Student s[], int &n, int roll) {
```

```
    int index = -1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (s[i].roll == roll) {
```

```
            index = i;
```

```
            break;
```

```
        }
```

```
    }
```

```

    if (index == -1) {
        cout << "Student Not Found!\n";
        return;
    }

    for (int i = index; i < n - 1; i++) {
        s[i] = s[i + 1];
    }

    n--;
    Student::count--;

    cout << "Student Deleted Successfully!\n";
}

void searchStudent(Student s[], int n, int roll) {
    for (int i = 0; i < n; i++) {
        if (s[i].roll == roll) {
            cout << "Student Found!\n";
            s[i].display();
            return;
        }
    }
    cout << "Student Not Found!\n";
}

int main() {
    Student s[100];
    int n = 0;
    int choice;

    while (true) {
        cout << "\n=== Student Database Menu ===\n";
    }
}

```

```
cout << "1. Add Student\n";
cout << "2. Delete Student\n";
cout << "3. Search Student\n";
cout << "4. Exit\n";
cout << "Enter your choice: ";
cin >> choice;

if (choice == 1) {
    addStudent(s, n);
}
else if (choice == 2) {
    int r;
    cout << "Enter Roll No to Delete: ";
    cin >> r;
    deleteStudent(s, n, r);
}
else if (choice == 3) {
    int r;
    cout << "Enter Roll No to Search: ";
    cin >> r;
    searchStudent(s, n, r);
}
else if (choice == 4) {
    cout << "Exiting Program...\n";
    break;
}
else {
    cout << "Invalid Choice!\n";
}
}
return 0;
}
```


Prac 6:

```
#include <iostream>
#include <cstring>
using namespace std;
class Student {
    char *name;
    int roll;
public:
    // Constructor
    Student(const char *n, int roll) {
        this->roll = roll;
        name = new char[strlen(n) + 1];
        strcpy(this->name, n);
        cout << "Constructor invoked. Memory allocated.\n";
    }
    // Display function
    void display() {
        cout << "Name: " << name << "\nRoll No: " << roll << endl;
    }
    // Destructor
    ~Student() {
        delete[] name;
        cout << "Destructor invoked. Memory freed.\n";
    }
};

int main() {
    Student *s1 = new Student("Rahul", 10);
    s1->display();

    delete s1; // Explicit deallocation
    return 0;
}
```

Prac 7:

```
#include <iostream>

#include <string>

using namespace std;

// Base class
class Person {
protected:
    string name;

public:
    Person(string n) : name(n) {}

    // Virtual function
    virtual void showDetails() {
        cout << "Person Name: " << name << endl;
    }
};

// Derived class (Level 1)
class Employee : public Person {
protected:
    int employeeId;
    string department;

public:
    Employee(string n, int id, string dept)
        : Person(n), employeeId(id), department(dept) {}

    // Overriding virtual function
    void showDetails() override {
        cout << "Employee Name: " << name << endl;
    }
};
```

```
        cout << "Employee ID: " << employeeId << endl;
        cout << "Department: " << department << endl;
    }
};
```

```
// Further derived class (Level 2)
```

```
class Manager : public Employee {
```

```
    int teamSize;
```

```
public:
```

```
    Manager(string n, int id, string dept, int tSize)
```

```
        : Employee(n, id, dept), teamSize(tSize) {}
```

```
// Overriding virtual function again
```

```
void showDetails() override {
```

```
    cout << "Manager Name: " << name << endl;
```

```
    cout << "Employee ID: " << employeeId << endl;
```

```
    cout << "Department: " << department << endl;
```

```
    cout << "Team Size: " << teamSize << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    // Base class pointer
```

```
    Person *ptr;
```

```
    // Employee object
```

```
    Employee e1("Sourav", 101, "HR");
```

```
    // Manager object
```

```
    Manager m1("Gaurav", 201, "IT", 8);
```

```
    // Pointing to Employee object
```

```
    ptr = &e1;
```

```
cout << "\n Details through base pointer (Employee object):" << endl;
ptr->showDetails(); // Calls Employee::showDetails()

// Pointing to Manager object
ptr = &m1;
cout << "\n Details through base pointer (Manager object):" << endl;
ptr->showDetails(); // Calls Manager::showDetails()

return 0;
}
```

Prac 8:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Base class
```

```
class Student {
```

```
protected:
```

```
    int rollNo;
```

```
    string name;
```

```
public:
```

```
    void getStudent() {
```

```
        cout << "Enter roll number: ";
```

```
        cin >> rollNo;
```

```
        cin.ignore(); // clear newline
```

```
        cout << "Enter name: ";
```

```
        getline(cin, name);
```

```
    }
```

```
    void showStudent() {
```

```
        cout << "Roll No: " << rollNo << endl;
```

```
        cout << "Name : " << name << endl;
```

```
    }
```

```
};
```

```
// Academic performance (virtual inheritance)
```

```
class Academic : public virtual Student {
```

```
protected:
```

```
    float academicMarks;
```

```
public:
```

```

void getAcademic() {
    cout << "Enter academic marks (out of 100): ";
    cin >> academicMarks;
}

void showAcademic() {
    cout << "Academic Marks: " << academicMarks << endl;
}
};

```

// Sports performance (virtual inheritance)

```

class Sports : public virtual Student {
protected:
    float sportsMarks;

public:
    void getSports() {
        cout << "Enter sports marks (out of 50): ";
        cin >> sportsMarks;
    }

    void showSports() {
        cout << "Sports Marks : " << sportsMarks << endl;
    }
};

```

// Result class (hybrid inheritance)

```

class Result : public Academic, public Sports {
    float total;
    char grade;

    char calculateGrade() {

```

```

float percentage = (total / 150.0f) * 100.0f;
if (percentage >= 75)
    return 'A';
else if (percentage >= 60)
    return 'B';
else if (percentage >= 50)
    return 'C';
else
    return 'D';
}

```

public:

```

void calculate() {
    total = academicMarks + sportsMarks;
    grade = calculateGrade();

    cout << "\n----- Student Performance Report ----- \n";
    showStudent();
    showAcademic();
    showSports();
    cout << "Total Marks : " << total << " / 150" << endl;
    cout << "Grade : " << grade << endl;
}
};

```

int main() {

Result r;

cout << "Enter student details:\n";

r.getStudent();

cout << "\nEnter academic performance:\n";

```
r.getAcademic();
```

```
cout << "\nEnter sports performance:\n";
```

```
r.getSports();
```

```
r.calculate();
```

```
return 0;
```

```
}
```


Prac 9:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

// Abstract base class
class Employee {
protected:
    string name;
    int id;

public:
    Employee(string n, int i) : name(n), id(i) {}

    // Pure virtual function
    virtual double calculatePay() const = 0;

    virtual void showInfo() const {
        cout << "ID : " << id << endl;
        cout << "Name : " << name << endl;
    }

    // Virtual destructor (good practice)
    virtual ~Employee() {}
};

// Hourly employee
class HourlyEmployee : public Employee {
    double hoursWorked;
    double hourlyRate;
```

```

public:
    HourlyEmployee(string n, int i, double hours, double rate)
        : Employee(n, i), hoursWorked(hours), hourlyRate(rate) {}

    double calculatePay() const override {
        return hoursWorked * hourlyRate;
    }

    void showInfo() const override {
        cout << "\n[Hourly Employee]\n";
        Employee::showInfo();
        cout << "Hours Worked : " << hoursWorked << endl;
        cout << "Hourly Rate : " << hourlyRate << endl;
    }
};

```

// Salaried employee

```

class SalariedEmployee : public Employee {
    double monthlySalary;

public:
    SalariedEmployee(string n, int i, double salary)
        : Employee(n, i), monthlySalary(salary) {}

    double calculatePay() const override {
        return monthlySalary;
    }

    void showInfo() const override {
        cout << "\n[Salaried Employee]\n";
        Employee::showInfo();
        cout << "Monthly Salary : " << monthlySalary << endl;
    }
};

```

```

    }
};

// Contract employee
class ContractEmployee : public Employee {
    double contractAmount;

public:
    ContractEmployee(string n, int i, double amount)
        : Employee(n, i), contractAmount(amount) {}

    double calculatePay() const override {
        return contractAmount;
    }

    void showInfo() const override {
        cout << "\n[Contract Employee]\n";
        Employee::showInfo();
        cout << "Contract Amount : " << contractAmount << endl;
    }
};

int main() {
    // Create employee objects
    HourlyEmployee h("Lokesh", 101, 160, 250);    // 160 hours * 250
    SalariedEmployee s("Shantanu", 201, 50000);    // fixed monthly salary
    ContractEmployee c("Ritesh", 301, 75000);    // contract amount

    // Array of base class pointers
    Employee* employees[3];
    employees[0] = &h;
    employees[1] = &s;

```

```
employees[2] = &c;
```

```
cout << fixed << setprecision(2);
```

```
cout << "----- Payroll System (Run-time Polymorphism) -----\\n";
```

```
for (int i = 0; i < 3; ++i) {
```

```
    employees[i]->showInfo();
```

```
    double pay = employees[i]->calculatePay(); // virtual call
```

```
    cout << "Calculated Pay : " << pay << "\\n";
```

```
}
```

```
return 0;
```

```
}
```

Prac 10:

```
#include <iostream>

#include <string>

#include <stdexcept>

using namespace std;

// Template-based sorting class
template <typename T>
class Sorter {
public:
    // Sort array in ascending order with basic validation
    static void sortArray(T* arr, int n) {
        if (arr == NULL) {
            throw invalid_argument("Null pointer passed to sortArray");
        }
        if (n <= 0) {
            throw invalid_argument("Invalid size passed to sortArray");
        }

        // Simple bubble sort
        for (int i = 0; i < n - 1; ++i) {
            for (int j = 0; j < n - 1 - i; ++j) {
                if (arr[j] > arr[j + 1]) {
                    T temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    // Print array elements
```

```

static void printArray(T* arr, int n) {
    if (arr == NULL || n <= 0) {
        cout << "Nothing to display (invalid array)." << endl;
        return;
    }

    for (int i = 0; i < n; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

};

int main() {
    try {
        // Integer array
        int intArr[] = {5, 2, 9, 1, 3};
        int n1 = sizeof(intArr) / sizeof(intArr[0]);

        cout << "Original int array: ";
        Sorter<int>::printArray(intArr, n1);

        Sorter<int>::sortArray(intArr, n1);

        cout << "Sorted int array : ";
        Sorter<int>::printArray(intArr, n1);

        // String array
        string strArr[] = {"Banana", "Apple", "Mango", "Grapes"};
        int n2 = sizeof(strArr) / sizeof(strArr[0]);

        cout << "\nOriginal string array: ";

```

```
Sorter<string>::printArray(strArr, n2);

Sorter<string>::sortArray(strArr, n2);

cout << "Sorted string array : ";
Sorter<string>::printArray(strArr, n2);

// Example of invalid input (to show exception handling)
cout << "\nTrying to sort with invalid size..." << endl;
Sorter<int>::sortArray(intArr, -5); // will throw exception
}
catch (const exception& e) {
    cerr << "Exception caught: " << e.what() << endl;
}

return 0;

}
```

Prac 11:

```
#include <iostream>

#include <vector>

#include <string>

using namespace std;

// Template-based custom vector wrapper
template <typename T>
class MyVector {
private:
    vector<T> data;

public:
    // Insert element at the end
    void insertElement(const T& value) {
        data.push_back(value);
    }

    // Delete element at given index
    void deleteAt(size_t index) {
        if (index >= data.size()) {
            cout << "Error: Index out of range, cannot delete." << endl;
            return;
        }
        data.erase(data.begin() + index);
    }

    // Search element, return index or -1
    int searchElement(const T& value) const {
        for (size_t i = 0; i < data.size(); ++i) {
            if (data[i] == value) {
                return static_cast<int>(i);
            }
        }
    }
};
```



```

    }
}
return -1;
}

// Display all elements
void display() const {
    if (data.empty()) {
        cout << "Vector is empty." << endl;
        return;
    }
    for (size_t i = 0; i < data.size(); ++i) {
        cout << data[i] << " ";
    }
    cout << endl;
}
};

int main() {
    // Integer example
    MyVector<int> intVec;
    intVec.insertElement(10);
    intVec.insertElement(5);
    intVec.insertElement(30);
    intVec.insertElement(20);

    cout << "Integer vector elements: ";
    intVec.display();

    cout << "Deleting element at index 1..." << endl;
    intVec.deleteAt(1); // delete 5
    cout << "After deletion: ";

```

```

intVec.display();

int keyInt = 30;
int posInt = intVec.searchElement(keyInt);
if (posInt != -1)
    cout << "Element " << keyInt << " found at index " << posInt << endl;
else
    cout << "Element " << keyInt << " not found." << endl;

// String example
MyVector<string> strVec;
strVec.insertElement("Apple");
strVec.insertElement("Mango");
strVec.insertElement("Banana");

cout << "\nString vector elements: ";
strVec.display();

cout << "Deleting element at index 0..." << endl;
strVec.deleteAt(0); // delete "Apple"
cout << "After deletion: ";
strVec.display();

string keyStr = "Banana";
int posStr = strVec.searchElement(keyStr);
if (posStr != -1)
    cout << "Element \" " << keyStr << "\" found at index " << posStr << endl;
else
    cout << "Element \" " << keyStr << "\" not found." << endl;

return 0;
}

```

Prac 12:

```
#include <iostream>
#include <stdexcept>
using namespace std;

// Custom exceptions
class InvalidAccountException : public exception {
public:
    const char* what() const throw() {
        return "Invalid account number!";
    }
};

class InsufficientFundsException : public exception {
public:
    const char* what() const throw() {
        return "Insufficient funds!";
    }
};

class BankAccount {
    int accountNumber;
    double balance;
public:
    BankAccount(int accNo, double initialBalance)
        : accountNumber(accNo), balance(initialBalance) {}

    void deposit(double amount) {
        if (amount <= 0) throw invalid_argument("Deposit must be positive.");
        balance += amount;
        cout << "Deposited: " << amount << ", Balance: " << balance << endl;
    }
}
```

```

void withdraw(double amount) {
    if (amount <= 0) throw invalid_argument("Withdrawal must be positive.");
    if (amount > balance) throw InsufficientFundsException();
    balance -= amount;
    cout << "Withdrawn: " << amount << ", Balance: " << balance << endl;
}

int getAccNo() const { return accountNumber; }
void showBalance() const { cout << "Balance: " << balance << endl; }
};

int main() {
    BankAccount acc(12345, 5000);

    cout<<"----- Bank Transaction System -----\\n";
    cout<<"Account No: 12345, Current Balance: 5000 \\n"<<endl;

    try {
        int inputAccNo;
        cout << "Enter account number: ";
        cin >> inputAccNo;
        if (inputAccNo != acc.getAccNo()) throw InvalidAccountException();

        acc.showBalance();
        acc.deposit(2000);
        acc.withdraw(1000);
        acc.withdraw(10000); // will throw exception
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << endl;
    }
    return 0;
}

```

